

2.3 Producing robust programs

Where this unit sits, what it covers, and how far

Where this unit sits

Qualification	GCSE Computer Science (9-1), specification code J277, from 2020
Component	Component 02: Computational thinking, algorithms and programming — 80 marks, 1 hour 30 minutes
What OCR says the component covers	computational thinking: algorithms, programming techniques, producing robust programs, computational logic and translators
This unit	topic 2.3 Producing robust programs
Checked	3 October 2026, at https://www.ocr.org.uk/qualifications/gcse/computer-science-j277-from-2020/specification-at-a-glance/

How far this has been checked. The topic numbers and titles below are the ones this course uses on its worksheets and slides. They were not re-checked against the published specification document when this map was built: the document could not be retrieved from this build environment. Check them against <https://www.ocr.org.uk/Images/558027-specification-gcse-computer-science-j277.pdf> before relying on this map for a scheme of work, and treat any finer subdivision as absent rather than assumed - no sub-point numbers are claimed anywhere in these documents, because none were verified.

Before it: 1.1 Systems architecture, 1.2 Memory and storage, 1.3 Computer networks, connections and protocols, 1.4 Network security, 1.5 Systems software, 1.6 Ethical, legal, cultural and environmental impacts of digital technology, 2.1 Algorithms, 2.2 Programming fundamentals

After it: 2.4 Boolean logic, 2.5 Programming languages and Integrated Development Environments, PY Python programming

The lesson sequence and the questions it answers

#	LESSON	THE BIG QUESTION	ACTIVITIES	MARKS
1	Security desk rp-l01	How can a programmer make sure a program copes with anything a user types?	13	19
2	Cyber defence HQ rp-l02	How can a program defend itself against careless users and deliberate attackers?	13	20
3	Code clinic rp-l03	What does code maintainability mean?	14	24
4	Bug hunt lab rp-l04	What are the different types of errors that can occur in a program?	13	20
5	Test lab rp-l05	What makes a good testing strategy?	14	23
6	Revision HQ 2.3 rp-l06	What makes a robust program?	19	25
7	End-of-topic test rp-l07-test	assesses the whole unit	0	0

Prior knowledge, vocabulary and skills

Assumed before the unit: the outcomes of 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 2.1, 2.2

Vocabulary introduced: Defensive design, Anticipating misuse, Authentication, Input validation, Maintainability, Naming conventions, Iterative testing, Final testing, Syntax error, Logic error, Testing, Logical error, Test data

What a pupil does, not only reads: bugline, code, match, mcq, multi, sort

Coverage matrix

Every outcome this unit states, and whether it is taught, practised and assessed **within this unit**. Full means all three; partial means some; not covered means none of the three could be traced.

How this was worked out. The trace is derived by matching the content words of an outcome against the station text, the activities and the exam questions of the unit, not authored by hand. The words each match was made on are in `build/coverage.json`. It shows where to look; it does not certify that the alignment is right, and a full row has not been checked by a person either.

OUTCOME	LESSON	TAUGHT	PRACTISED	ASSESSED	COVERAGE
know what is meant by defensive design when writing programs <code>rp-l01-o1</code>	rp-l01	yes	yes	yes	Full
understand why input validation is necessary <code>rp-l01-o2</code>	rp-l01	yes	yes	yes	Full
know a range of validation techniques used to write robust programs <code>rp-l01-o3</code>	rp-l01	yes	yes	yes	Full
know what is meant by defensive design considerations <code>rp-l02-o1</code>	rp-l02	yes	yes	yes	Full
know a range of problems that can occur when a program is running <code>rp-l02-o2</code>	rp-l02	yes	yes	yes	Full
understand some authentication techniques programmers use <code>rp-l02-o3</code>	rp-l02	yes	yes	yes	Full
know why creating code that is easy to read is important with large projects <code>rp-l03-o1</code>	rp-l03	yes	yes	—	Partial
understand what programmers can do to make their code more readable <code>rp-l03-o2</code>	rp-l03	yes	yes	yes	Full
understand how to refine algorithms to make them more robust <code>rp-l03-o3</code>	rp-l03	yes	yes	yes	Full
know four reasons why a program should be tested <code>rp-l04-o1</code>	rp-l04	yes	yes	yes	Full
know what iterative testing is <code>rp-l04-o2</code>	rp-l04	yes	yes	yes	Full
know what final/terminal testing is <code>rp-l04-o3</code>	rp-l04	yes	yes	yes	Full
know what a syntax error is <code>rp-l04-o4</code>	rp-l04	yes	yes	yes	Full

OUTCOME	LESSON	TAUGHT	PRACTISED	ASSESSED	COVERAGE
know what a logic error is rp-104-o5	rp-l04	yes	yes	yes	Full
understand that just because a program works doesn't mean it works for all inputs rp-105-o1	rp-l05	yes	yes	yes	Full
understand that test data should include normal, boundary, invalid and erroneous data rp-105-o2	rp-l05	yes	yes	yes	Full
understand how robust programs are made rp-106-o1	rp-l06	yes	yes	yes	Full
know which lessons I'm secure on and which to revise rp-106-o2	rp-l06	yes	yes	—	Partial

16 full · **2 partial** · **0 not covered**, of 18 outcomes.

Gaps, assumptions and overlap

- **rp-l03-o1** know why creating code that is easy to read is important with large projects — nothing in this unit assesses it

What could not be traced

The wall text of rp-103, rp-104 is painted into the 360 image and held nowhere as text, so an outcome in those lessons lesson can only be traced through its activities and its exam questions. Read the rows above for them with that in mind.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.